5.2.4. Operations on std::atomic<T*>: pointer arithmetic

The atomic form of a pointer to some type `T` is `std::atomic<T*>`, just as the atomic form of `bool` is `std::atomic<bool>`. The interface is essentially the same, although it operates on values of the corresponding pointer type rather than `bool` values. Just like `std::atomic<bool>`, it's neither copy-constructible nor copy-assignable, although it can be both constructed and assigned from the suitable pointer values. As well as the obligatory `is_lock_free()` member function, `std::atomic<T*>` also has `load()`, `store()`, `exchange()`, `compare_exchange_weak()`, and `compare_exchange_strong()` member functions, with similar semantics to those of `std::atomic<bool>`, again taking and returning `T*` rather than `bool`.

The new operations provided by `std::atomic<T*>` are the pointer arithmetic operations. The basic operations are provided by the `fetch_add()` and `fetch_sub()` member functions, which do atomic addition and subtraction on the stored address, and the operators `+=` and `-=`, and both pre- and post-increment and decrement with `++` and `--`, which provide convenient wrappers. The operators work just as you'd expect from the built-in types: if `x` is `std::atomic<Foo*>` to the first entry of an array of `Foo` objects, then `x+=3` changes it to point to the fourth entry and returns a plain `Foo*` that also points to that fourth entry. `fetch_add()` and `fetch_sub()` are slightly different in that they return the original value (so `x.fetch_add(3)` will update `x` to point to the fourth value but return a pointer to the first value in the array). This operation is also known as *exchange-and-add*, and it's an atomic read-modify-write operation, like `exchange()` and `compare_exchange_weak()`/`compare_exchange_strong()`. Just as with the other operations, the return value is a plain `T*` value rather than a reference to the `std::atomic<T*>` object, so that the calling code can perform actions based on what the previous value was:

```
class Foo{};
Foo some_array[5];
std::atomic<Foo*> p(some_array);
Foo* x=p.fetch_add(2);
assert(x==some_array);
assert(p.load()==&some_array[2]);
x=(p-=1);
assert(x==&some_array[1]);
assert(p.load()==&some_array[1]);
```

Add 2 to p and return old value

Subtract 1 from p and return new value

The function forms also allow the memory-ordering semantics to be specified as an additional function call argument:

```
p.fetch_add(3,std::memory_order_release);
```

Because both `fetch_add()` and `fetch_sub()` are read-modify-write operations, they can have any of the memory-ordering tags and can participate in a *release sequence*. Specifying the ordering semantics isn't possible for the operator forms, because there's no way of providing the information: these forms therefore always have `memory_order_seq_cst` semantics.